

NEROLITH

Architecture Research & Evolution Blueprint
Geospatial Intelligence · Multi-Agent AI · Digital Twins · Climate-Tech

1. Architecture Evolution: Flood Sim → Planet-Scale Intelligence

Your current 3-layer stack is a solid foundation, but it is architecturally bounded. Physics → AI → Viz is a pipeline, not a platform. To reach planet scale you need to introduce four new architectural primitives: **a Data Fabric, an Ontology Layer, a Temporal Engine, and a Federation Protocol.**

1.1 The 7-Layer Target Architecture

L1 — Data Ingestion Fabric

Unified ingest for satellite imagery (Sentinel-2/Landsat), IoT sensors, weather APIs, LiDAR point clouds, OpenStreetMap, and real-time stream data (Kafka/MQTT). Every data source is normalised to a common geospatial schema before touching any algorithm.

L2 — Physics Engine (your current C++ core)

Deterministic simulation stays here. This layer must become modular: plug-in algorithm slots (swap D8 for MFD, swap Green-Ampt for SWMM) without touching any other layer. Expose via gRPC, not raw DLL calls.

L3 — Spatial Knowledge Graph

Terrain regions, infrastructure assets, administrative boundaries, historical flood events — all as nodes/edges in a property graph (Neo4j or custom). This is the memory of the system.

L4 — Multi-Agent Intelligence Layer (your current Python agents, extended)

Agents query the knowledge graph, run ML inference, coordinate via a message bus. This layer scales horizontally: 10 agents for a district, 10,000 for a country.

L5 — Temporal Reasoning Engine

Maintains time-series state for every spatial entity. Answers queries like: 'What was the flow accumulation at node X 3 hours before the last flood event?' Backed by TimescaleDB or InfluxDB with geospatial extensions.

L6 — Decision & Action Layer

The layer that actually does things: issues alerts, triggers evacuation route APIs, dispatches to government dashboards, calls infrastructure management systems. Rule engine + RL policy runs here.

L7 — Visualization & Interface Layer (your current Qt/OpenGL, extended)

Web dashboard (React/Deck.gl) for agencies, desktop app for engineers, REST/GraphQL API for third-party integration, mobile push for citizen alerts.

1.2 The Critical New Abstraction: Spatial Entity Model

Every 'thing' in Nerolith — a river segment, a bridge, a city block, a rainfall cell — must be modelled as a Spatial Entity with four mandatory properties: geometry, temporal state history, risk score, and agent ownership. This single abstraction lets every layer talk the same language. Without it, your system will drown in ad-hoc data structures as

complexity grows.

1.3 Federation: From City to Planet

Planet-scale means you cannot run one monolithic simulation. You need a federation protocol: each Nerolith deployment (a city, a watershed) exposes a standardised API. A meta-coordinator can aggregate outputs from 1,000 city-level instances. Think of it as a WRF-style domain nesting, but for AI agents. This is how you go from 'flood tool' to 'planetary intelligence system.'

2. Advanced AI Integration

The key insight is that different AI paradigms solve different parts of your problem. Do not try to use one model for everything. Here is the precise mapping:

2.1 Classical ML — Regression & Forecasting

- Where it fits: Physics Engine output post-processing and sensor calibration.
- What it solves: Your physics engine produces depth/velocity grids. ML corrects systematic errors by learning residuals between simulated and observed flood extents (using historical satellite imagery as ground truth). This is Physics-ML blending — you already have a slot for it in your engine.
- Specific models: XGBoost for feature-based flood susceptibility scoring per grid cell; LSTM/Transformer for 6–72 hour rainfall and river level forecasting from station time series.
- Data requirement: 5–10 years of historical rainfall + gauge data. Available free from NOAA, IMD, and Copernicus Climate Data Store.

2.2 Deep Learning — Spatial CNNs and Graph Neural Networks

- Spatial CNNs (U-Net architecture): Treat your DEM + rainfall grid as a multi-channel image. Train a U-Net to predict flood inundation maps directly from inputs. This runs 1000x faster than your physics engine with ~85–90% accuracy — use it for rapid screening before committing to full physics simulation.
- Where it fits: Parallel fast-path in L2, feeds L4 agents with quick-estimate grids.
- Graph Neural Networks (GNN): Your terrain graph is the natural input. A GNN operating on your flow network can predict which downstream nodes will be affected given an upstream event, propagating uncertainty through the graph topology. This is something your physics engine cannot do probabilistically.
- Where it fits: L3 Knowledge Graph + L4 agents. Agents use GNN inference instead of waiting for full physics propagation.
- Specific architecture: GraphSAGE or GAT (Graph Attention Network) trained on historical flood propagation sequences across your terrain graph.

2.3 Reinforcement Learning — Adaptive Decision Systems

- Where it fits: Exclusively in L6, the Decision & Action Layer.
- What it solves: Static rule-based alert systems are fragile. An RL policy learns the optimal action (issue alert, recommend evacuation, close road) given the current system state, balancing false-positive costs against missed-warning costs.
- Setup: State = vector of agent observations + infrastructure status + weather forecast. Action space = discrete (alert level 1/2/3, evacuation zones A/B/C, resource dispatch). Reward = weighted combination of lives protected, economic damage avoided, false alarm rate.

- Training environment: Your physics engine IS the simulation environment for RL training. This is huge — most RL systems lack a realistic simulator. You already have one.
- Practical path: Start with PPO (Proximal Policy Optimization) on synthetic flood scenarios generated by your engine. Fine-tune on historical events.

2.4 Generative AI — LLM + Reasoning Agents

- Where it fits: Report Agent (already planned) + a new Query Interface.
- Report Agent: LLaMA 3.1 (you have this) generates natural language situation reports from structured agent outputs. Add structured output schemas so the LLM fills a JSON template first, then narrates — this prevents hallucination of numerical data.
- Query Interface: Let an engineer ask 'What caused the 2023 Yamuna flood and how would the same rainfall affect the new metro bridge?' — the LLM decomposes this into graph queries + physics simulation calls + historical lookup, then synthesises the answer. This is the GraphRAG integration (covered in Section 4).
- Multimodal extension: GPT-4V / LLaMA-Vision to interpret satellite imagery and annotate flood extents, feeding directly into your knowledge graph as observations.

3. Multi-Agent Evolution: Observers → Planners

3.1 The Three-Generation Model

Gen 1 (current): Observer agents. Consume physics output, produce alerts. Stateless.

Gen 2 (next): Predictive agents. Maintain local state, run lightweight ML inference, share predictions peer-to-peer. Each region agent owns a forecast model for its patch.

Gen 3 (target): Planning agents. Reason about future states, negotiate resource allocation, run counterfactual simulations ('what if we close this dam gate now?'), and propose infrastructure interventions.

3.2 Swarm Intelligence vs Decentralised Coordination

You should NOT use classic swarm intelligence (ACO, PSO) for Nerolith. Those algorithms optimise continuous numerical problems. Your problem is spatial-temporal decision-making with heterogeneous agents. The right answer is Decentralised Partially Observable Markov Decision Processes (Dec-POMDP) combined with belief propagation on your terrain graph.

- Each region agent has a local belief state about its patch (derived from physics output + ML forecast + sensor data).
- Agents share belief updates with neighbours via the terrain graph edges (flow-connected regions communicate; isolated regions do not).
- The Coordinator agent runs global consensus — it does not simulate, it aggregates regional beliefs and resolves conflicts (two agents disagree on flood extent boundary).
- This is mathematically equivalent to belief propagation on a factor graph, which is well-studied and has efficient algorithms.

3.3 Predictive Agent Architecture

Each region agent should run a local LSTM or Kalman Filter that maintains a 6-hour forecast for its region's flood depth. When its forecast diverges from the physics engine output by more than a threshold, it flags the discrepancy to the Coordinator. This divergence is an early warning signal — it means either the physics simulation is missing a local

factor (blocked drain, soil saturation anomaly) or the sensor data is corrupted.

3.4 Planning Agents: The Key Capability

Planning agents require a world model. In your architecture, the world model is the combination of your physics engine + knowledge graph. A planning agent uses Monte Carlo Tree Search (MCTS) over simulated future states to find optimal intervention sequences. This is computationally expensive — run planning agents only when triggered by an alert threshold, not continuously. The planning horizon should be 24–72 hours.

4. Graph + Knowledge System

4.1 Terrain Graph → Full Knowledge Graph

Your terrain graph (regions as nodes, water flow as edges) is the skeleton. The full knowledge graph adds three new node types and four new edge types:

Infrastructure nodes: bridges, dams, culverts, roads, hospitals, power stations — each with capacity, vulnerability score, and maintenance history.

Event nodes: historical flood events, construction events, land-use changes — timestamped and linked to the terrain nodes they affected.

Regulatory nodes: flood zone designations, building codes, evacuation plans — linked to administrative boundary nodes.

Edge: DRAINS INTO: terrain → terrain, weighted by D8 flow accumulation.

Edge: THREATENS: flood region → infrastructure node, weighted by depth × duration.

Edge: CAUSED_BY: event → event (historical causality chain).

Edge: MITIGATES: infrastructure → flood region (dam capacity, levee protection radius).

4.2 GraphRAG for Geospatial Reasoning

GraphRAG (Graph-Retrieval Augmented Generation) is exactly the right pattern here. Standard RAG retrieves text chunks; GraphRAG retrieves subgraphs. In your case:

- An engineer asks: 'What is the risk to the NH-58 bridge given a 100-year return period flood?'
- Step 1 — Entity extraction: LLM identifies 'NH-58 bridge' and '100-year flood' as entities.
- Step 2 — Graph traversal: System queries the knowledge graph, retrieves the bridge node, its upstream terrain nodes, historical flood events at those nodes, and the THREATENS edges.
- Step 3 — Physics query: The retrieved subgraph triggers a targeted physics simulation with 100-year return period rainfall as input.
- Step 4 — Synthesis: LLM narrates the simulation output + historical context into a structured risk assessment report.
- This is the core product differentiator. ArcGIS cannot do this. QGIS cannot do this.

4.3 Graph Schema Implementation

- Use Neo4j for the knowledge graph in development (Cypher queries, built-in graph algorithms).
- For production at scale, consider Apache AGE (PostgreSQL extension) — keeps geospatial data and graph data in one system with PostGIS integration.
- Every node must carry: node_id, geometry (WKT), created_at, updated_at, source, confidence_score.
- Run Neo4j GDS (Graph Data Science) library for centrality analysis — high-betweenness nodes in your flow graph are critical infrastructure chokepoints.

5. Digital Twin Vision

5.1 What a Real Digital Twin Requires

A digital twin is not a simulation. It is a continuously synchronised computational replica of a physical system. The key word is continuously. Your current architecture is event-driven (run simulation on demand). A true digital twin requires four components that you do not yet have:

Continuous data ingestion: real-time sensor streams update the twin's state automatically, not on user request.

State synchronisation protocol: a formal mechanism to detect when the twin diverges from physical reality and trigger recalibration.

Uncertainty quantification: every state variable carries a confidence interval, not a point estimate.

Bidirectional coupling: the twin can influence physical systems (send alerts, recommend valve adjustments) and receive feedback on whether those actions were taken.

5.2 Data Sources for City-Scale Twin

Satellite (free): Sentinel-1 SAR (flood mapping, 6-day revisit), Sentinel-2 multispectral (land use, 5-day revisit), SRTM/ASTER DEM (30m resolution).

Satellite (commercial): Planet Labs (daily 3m imagery), Maxar (50cm, event-triggered), Capella Space (SAR, 1m, any weather).

IoT / Ground sensors: river gauges (CWC in India), rain gauges, soil moisture sensors, smart drain level sensors, traffic cameras with CV-based flood detection.

Crowdsourced: Twitter/X flood reports (NLP classification), Google Waze road closures, WhatsApp community groups via bot integration.

Infrastructure: SCADA systems from water utilities, dam telemetry, power grid status APIs.

5.3 Continuous Learning Loop

- After every real flood event: extract satellite-derived flood extent as ground truth.
- Compare against Nerolith's prediction at T+0 of the event.
- Compute prediction error spatially — which regions were over/under-predicted.
- Fine-tune ML models (CNN, LSTM) on the error signal. Update knowledge graph with new event node + observed impact data.
- Recalibrate physics engine parameters (Manning's n, soil infiltration rates) using Bayesian optimisation against observed data.
- This loop turns every real event into a training example. After 3–5 years, Nerolith trained on a city's history will outperform any generic model.

6. Product + Startup Direction

6.1 Market Positioning

Do not position against ArcGIS/QGIS. They are GIS tools. Nerolith is a decision system. Your competitive frame should be: 'Early Warning Intelligence Platform' not 'Flood Simulation Software.' The buyer is not a GIS analyst. The buyer is a District Collector, a Smart City CEO, or a P&C; Insurance underwriter.

6.2 Go-To-Market Segments (Ranked by Ease of Sale)

- 1. State Disaster Management Authorities (SDMAs):** India has 28 SDMAs with annual budgets for technology. They are actively looking for AI-based early warning systems post-2023 floods. NDMA has a formal vendor empanelment process. One SDMA contract = 2–5 Cr ARR.
- 2. Infrastructure EPC Companies:** L&T, Afcons, NCC build bridges and roads in flood-prone areas. They need site-specific flood risk assessments for project DPRs. Sell as a per-project API report. High volume, fast sales cycle.
- 3. Smart City Mission projects:** 100 smart cities have IoT budgets. Nerolith integrates with existing sensor networks and adds AI intelligence. Position as 'Flood Intelligence Module' for existing city OS.
- 4. Reinsurance / Climate Risk:** Swiss Re, Munich Re, and Indian reinsurers need hyper-local flood exposure data. This is a global market. Sell probabilistic flood maps as data product via API. Highest revenue potential, longest sales cycle.

6.3 Business Model

- Tier 1 — Data API: Per-query pricing for flood risk scores, probabilistic maps, historical event data. Target: insurers, developers, consultants.
- Tier 2 — SaaS Dashboard: Annual subscription for real-time monitoring + alerts + LLM-generated reports. Target: SDMAs, municipal corporations.
- Tier 3 — Enterprise / On-Premise: Full deployment of the engine + agents in the client's infrastructure (required for sensitive government data). One-time licence + annual maintenance. Target: NDMA, state governments.
- Tier 4 — Digital Twin as a Service: Continuous city twin with dedicated compute, custom sensor integration, SLA-backed uptime. Premium pricing. Target: smart cities, large utilities.

7. Next-Level Features

F1 — Real-Time Probabilistic Evacuation Routing

Not 'evacuate zone A.' Nerolith computes optimal evacuation routes for every neighbourhood simultaneously, accounting for road flooding state (from physics engine), traffic density (from city APIs), and shelter capacity. Routes update every 15 minutes as flood evolves. Output: a live evacuation graph pushed to emergency services and citizen apps.

F2 — Infrastructure Failure Cascade Prediction

Model infrastructure as a dependency graph. When a culvert's capacity is exceeded, the GNN predicts which downstream roads flood, which bridges lose structural margin, which electricity substations are at risk. Issue pre-failure alerts 2–6 hours ahead. This saves lives and prevents the cascading failures seen in every major Indian flood.

F3 — Counterfactual Infrastructure Planning

An engineer inputs a proposed new drainage system. Nerolith runs the physics simulation with and without the intervention across 50 historical flood scenarios. Output: quantified reduction in flood extent, damage avoided, cost-benefit ratio. This turns Nerolith into a mandatory tool for infrastructure DPR preparation.

F4 — Multi-Hazard Compound Event Modelling

Real disasters are compound events. A cyclone causes rainfall (flood) + wind damage (structural failure) + storm surge (coastal inundation) simultaneously. Nerolith models the joint probability and compound impact. Add landslide risk via slope stability analysis on the DEM + rainfall trigger. Add heat stress index for evacuation route viability. No existing commercial tool does this at city scale.

F5 — Climate Scenario Stress Testing

Feed IPCC SSP2.6 / SSP5.8.5 projected rainfall distributions into the physics engine. For every infrastructure asset, compute its flood exceedance probability in 2030, 2050, 2080. Output: a climate vulnerability atlas. This is the product for P&C; insurers and infrastructure financiers who need 30-year risk horizons.

F6 — Autonomous Sensor Placement Optimisation

Given a city's existing sensor network, Nerolith identifies which additional gauge locations would maximally reduce prediction uncertainty (information-theoretic optimal placement). This is a direct sale to municipal corporations upgrading their monitoring networks.

F7 — Post-Disaster Damage Assessment Automation

After a flood event, ingest SAR satellite imagery. Run change detection against pre-event baseline. Auto-classify damage by type (inundated building, damaged road, collapsed structure). Generate NDMA-format damage assessment report in under 1 hour. Currently this takes government teams 2–4 weeks manually.

F8 — Urban Growth Flood Risk Projection

Connect to land-use change data (urban expansion polygons). Model how planned development changes flood dynamics over 10 years — increased impervious surface, reduced retention. This is the tool for urban planners and municipal master plan preparation. Turns Nerolith from a reactive tool into a prospective one.

8. Honest Critique

8.1 What Is Weak Right Now

- **No real data validation pipeline.** Your physics engine takes DEM + rainfall as input. In production, both are noisy, missing, or misaligned in projection. You have no layer that cleans, validates, and reprojects input data. This will cause silent failures that look like physics bugs.
- **Agent coordination is undefined at the protocol level.** You describe agents as observers with roles, but there is no formal message schema, no conflict resolution protocol, and no ordering guarantee for agent updates. At 50+ agents this will produce race conditions and inconsistent global state.
- **The LLM Report Agent has no guardrails against hallucinating numbers.** If LLaMA 3.1 produces a situation report saying 'depth of 4.7m' when the physics engine computed 1.2m, that is a dangerous failure in a life-safety system. Implement structured output schemas and numeric grounding before deploying this.
- **No uncertainty quantification anywhere.** Your system outputs point estimates. Every decision-maker needs confidence intervals. 'Flood depth: 1.2m ± 0.4m' is actionable. 'Flood depth: 1.2m' from a model with unknown error is not trustworthy.

8.2 What Will Break at Scale

- The DLL architecture will not scale horizontally. A DLL is a single-process, single-machine artefact. When you need to run 200 simultaneous simulations for different cities, you need the physics engine exposed as a stateless service (containerised, gRPC interface). Redesign this before it becomes load-bearing.
- WebSocket to Qt will become a bottleneck. At high grid resolutions (1m DEM for a 50km² city) the data volume per tick will exceed what a single WebSocket can push without compression and delta encoding. Implement binary protocol (FlatBuffers or Protocol Buffers) with delta-only updates, not full grid dumps.
- The knowledge graph will degrade without a graph maintenance strategy. As events accumulate, old nodes become stale. You need TTL policies, graph pruning jobs, and a versioning strategy for node state.
- The multi-agent system has no fault isolation. If one region agent crashes or enters an infinite reasoning loop, it should not take down the coordinator. Each agent needs a watchdog, a timeout, and a fallback to last-known-good state.

8.3 What Is Over-Engineered or Unnecessary Right Now

- Reinforcement Learning at this stage. You do not have the historical data volume or the validated simulation fidelity needed to train a trustworthy RL policy. Build rule-based decision logic first. Add RL in year 2 when you have real event data.
- Planet-scale federation before you have one validated city deployment. Every system that tried to build planetary scale before city scale failed. Nail one watershed, one city, one SDMA. Then scale.
- The Qt desktop app as the primary interface for government users. Government agencies will not install a C++ desktop app. They need a web browser dashboard. Qt is the right choice for the engineering-grade simulation tool for your technical users, but you need a web tier for decision-makers and officials.
- Multiple LLM integrations simultaneously. Pick one inference path — LLaMA 3.1 locally or an API-based model. Trying to run multiple LLM backends adds complexity with no user-visible benefit at this stage.

8.4 The Honest Bottom Line

Nerolith has a genuinely differentiated core: a physics engine built by someone who understands the hydrology, paired with an agent architecture that is conceptually sound. Most 'AI flood tools' are pure ML black boxes with no physical grounding. Yours is not. That is a real moat.

The risk is architectural fragmentation — too many layers, too many technologies, not enough integration discipline. The systems that win in this space (Deltares, DHI) are boring under the hood: one unified data model, one simulation core, incremental capability additions over decades. Your ambition is right. Your pace needs to match your validation, not your vision.

The sequence that maximises your probability of success: (1) Get one real watershed running end-to-end with real DEM data and validation against one historical flood event. (2) Build the knowledge graph around that watershed. (3) Add GraphRAG query interface — this is your demo killer feature. (4) Find one SDMA or municipal body willing to pilot. (5) Scale from that anchor. Everything else follows.